

Manual Profesional de Bash & Shell Scripting

Desde Fundamentos de Terminal hasta Automatización Avanzada en Linux

Autor original: Apuntes de Estudio | Edición y Ampliación Técnica Profesional

1. Conceptos Fundamentales: Terminal, Shell y Bash

Para trabajar de manera efectiva en entornos de línea de comandos, es vital comprender la separación de responsabilidades entre la interfaz, el intérprete y el lenguaje utilizado.

Componente	Definición y Rol Técnico	Ejemplos Comunes
La Terminal (El Entorno/Monitor)	Interfaz gráfica o emulador de consola que captura la entrada del usuario (teclado) y dibuja la salida (pantalla). No procesa lógica por sí sola.	Warp, Windows Terminal, Alacritty, GNOME Terminal, iTerm2.
La Shell (El Intérprete/Traductor)	El programa subyacente que recibe los comandos de texto de la terminal, interactúa con el Kernel del Sistema Operativo para ejecutarlos y devuelve la respuesta.	Sh, Bash, Zsh, Fish, PowerShell.
Bash (El Lenguaje/Estándar)	<i>Bourne Again SHell</i> . Es un modelo o implementación específica de Shell, altamente compatible con la norma POSIX y el estándar <i>de facto</i> en distribuciones Linux y servidores productivos.	Bash v4/v5 (Estándar en Ubuntu, Debian, RHEL).

💡 La Analogía del Restaurante

Imagina un restaurante: La **Terminal** es la mesa donde te sientas (el punto de contacto). La **Shell** es el camarero (quien toma tu pedido y lo lleva a la cocina/kernel). **Bash** es el idioma específico (español, inglés) en el que está escrito el menú y hablas con el camarero.

Entornos de Trabajo Recomendados

- **WSL2 (Windows Subsystem for Linux)**: Capa de virtualización ligera en Windows que permite ejecutar un Kernel de Linux real (ej. Ubuntu) directamente sin el consumo de memoria de una máquina virtual tradicional.
- **Warp Terminal**: Emulador de terminal moderno impulsado por IA con autocompletado avanzado, selección estilo IDE y bloc de comandos.
- **Rutas de Acceso en WSL**: Tu usuario de Linux reside en `/home/usuario`, mientras que tus discos de Windows se montan bajo `/mnt/c/Users/TuUsuario/`.

2. Anatomía de un Comando y Comandos Básicos

La estructura general de una instrucción en la línea de comandos sigue la sintaxis:

```
comando [opciones] [argumentos]
```

Ejemplo:

```
ls -lh /var/log
```

- **Comando**: El programa ejecutable o función interna de la Shell (ej. `ls`, `grep`).

- **Opciones (Flags):** Modifican el comportamiento del comando. Suelen iniciar con un guion corto (`-l`) o doble guion (`--long`).
- **Argumentos:** Los datos sobre los cuales opera el comando (ficheros, directorios, cadenas).

Comandos Esenciales de Navegación e Información

```
whoami      # Muestra el nombre del usuario actual.
pwd         # Print Working Directory: muestra la ruta absoluta actual.
hostname    # Muestra el nombre del equipo/servidor en la red.
uname -a    # Muestra información detallada del Kernel de Linux.
uptime      # Tiempo que el sistema lleva encendido y carga media de CPU.
date        # Muestra o establece la fecha y hora del sistema.
cal         # Despliega un calendario interactivo en consola.
clear       # Limpia la pantalla de la terminal (Shortcut: Ctrl + L).
```

Uso Avanzado de `ls` (Listado de Archivos)

```
ls          # Listado simple.
ls -l       # Formato largo: permisos, propietarios, tamaño y fecha.
ls -la      # Muestra también archivos ocultos (los que inician con punto .).
ls -lh      # "Human Readable": muestra tamaños en KB, MB o GB.
ls -lt      # Ordena los archivos por fecha de última modificación.
```

Obtención de Ayuda y Documentación

- `man` : Abre el manual oficial POSIX completo (ej. `man ls`). Navega con flechas y sal con `q`.
- `--help` : Muestra una ayuda rápida con opciones válidas. Ideal para binarios modernos (ej. `python3 --help`).

3. Estructura de Archivos en Linux y Manipulación

Linux sigue la norma FHS (*Filesystem Hierarchy Standard*), donde todo cuelga de la raíz `/`.

Directorio	Propósito Principal
<code>/</code> (Raíz)	El nodo principal del cual dependen todos los subdirectorios y discos montados.
<code>/home</code>	Carpetas personales de los usuarios del sistema (ej. <code>/home/andres/</code>).
<code>/etc</code>	Archivos de configuración globales del sistema operativo y aplicaciones.
<code>/bin</code> / <code>/usr/bin</code>	Ejecutables e hiper-comandos esenciales para todos los usuarios.
<code>/var</code>	Datos variables: logs del sistema (<code>/var/log</code>), bases de datos y colas.
<code>/tmp</code>	Archivos temporales borrados automáticamente al reiniciar el sistema.

Manipulación de Ficheros y Directorios

```
# Creación y Eliminación
mkdir -p proyectos/python/src # Crea directorios anidados recursivamente.
touch index.html              # Crea un archivo vacío o actualiza su timestamp.
rmdir carpeta_vacia           # Elimina únicamente directorios sin contenido.

# Copiado Profesional (cp)
cp archivo.txt copia.txt      # Copia simple de un archivo.
cp -r carpeta/ nueva_carpeta/ # Copia recursiva (crea nuevos timestamps/permisos).
cp -a carpeta/ clon_exacto/    # Copia en modo ARCHIVO: preserva permisos, propietario y
                                timestamps originales.

# Mover y Renombrar (mv)
mv notas.txt /tmp/            # Mueve un archivo a otro directorio.
mv viejo.txt nuevo.txt        # Renombra un archivo en el mismo directorio.

# Eliminación Segura y Forzada (rm)
rm archivo.txt                # Borra un archivo regular.
rm -ri carpeta_sensible/      # Interactivo: solicita confirmación por cada archivo.
rm -rf carpeta_eliminar/      # Recursivo y forzado. ¡Usar con extrema precaución!
```

Comodines (Wildcards)

- `*` (Asterisco): Coincide con 0 o más caracteres. Ej: `rm *.log` borra todos los logs.
- `?` (Interrogación): Coincide con exactamente 1 carácter. Ej: `ls foto_?.jpg` busca `foto_01.jpg`.
- `[ ]` (Corchetes): Coincide con rangos o conjuntos. Ej: `ls doc_[a-z].txt`.

4. Búsqueda y Filtrado Avanzado: `find`, `grep` y Visualización

Lectura de Archivos

- `cat archivo.txt`: Imprime todo el archivo de golpe en pantalla (ideal para archivos pequeños).
- `less archivo.log`: Abre un visor paginado. Navega con flechas/espacio y busca con `/patrón`.
- `head -n 20 archivo.txt`: Muestra las primeras 20 líneas.
- `tail -n 20 archivo.txt`: Muestra las últimas 20 líneas.
- `tail -f /var/log/syslog`: ****Modo seguimiento en tiempo real****. Muestra nuevas líneas a medida que se escriben en el log.

El Comando `find` (Búsqueda en Sistema de Archivos)

```
# Buscar por nombre (insensible a mayúsculas con -iname)
find . -iname "*.py"

# Filtrar por tipo (f = archivo, d = directorio) y tamaño
find /var/log -type f -size +50M

# Búsqueda con ejecución automática (-exec)
# Busca todos los .tmp y los elimina. {} recibe cada archivo encontrado y \; finaliza el
comando.
find . -type f -name "*.tmp" -exec rm {} \;
```

El Comando `grep` (Filtro de Contenido por Expresión Regular)

```
grep "ERROR" /var/log/app.log           # Muestra líneas que contienen "ERROR".
grep -i "error" app.log                 # Insensible a mayúsculas/minúsculas.
grep -v "DEBUG" app.log                 # Invertir búsqueda: muestra líneas que NO contienen
DEBUG.
grep -rn "DB_PASSWORD" /etc/           # Recursivo (-r) indicando número de línea (-n).
```

Redirecciones y Tuberías (Pipes)

Operador	Nombre	Función
>	Redirección de Salida (Sobrescribir)	Envía la salida de un comando a un archivo, borrando el contenido previo.
>>	Redirección de Salida (Anexar)	Añade la salida al final del archivo sin borrar lo existente.
<	Redirección de Entrada	Usa el contenido de un archivo como entrada para un comando.
2>	Redirección de Errores (STDERR)	Redirige únicamente los mensajes de error a un archivo.
	Tubería (Pipe)	Conecta la salida de un comando directamente con la entrada de otro.

```
# Ejemplo Combinado Avanzado:
ps aux | grep python | wc -l > procesos_python.txt
# Cuenta cuántos procesos de Python se están ejecutando y guarda la cifra en un archivo.
```

5. Editores de Texto de Consola: Nano, Vim y Neovim

Nano: El Editor Directo

Indicado para ediciones rápidas. La tecla `^` representa `Ctrl`.

- `Ctrl + O`: Guardar el archivo (Confirmar con Enter).
- `Ctrl + X`: Salir del editor.
- `Ctrl + W`: Buscar texto dentro del fichero.
- `Ctrl + K` / `Ctrl + U`: Cortar línea / Pegar línea.

Vim / Neovim: El Editor Basado en Modos

 **Regla Fundamental de Vim**

Vim no permite escribir al abrirse. Inicia en **Modo Normal**. Presiona `i` para pasar al **Modo Insertar**. Para volver al Modo Normal, presiona siempre `Esc`.

Modo	Tecla de Acceso	Propósito / Comandos Esenciales
Normal	<code>Esc</code>	Navegación y comandos rápidos: <code>dd</code> (borrar línea), <code>yy</code> (copiar línea), <code>p</code> (pegar), <code>u</code> (deshacer), <code>Ctrl+r</code> (rehacer). Movimiento: <code>h, j, k, l</code> , <code>0</code> (inicio línea), <code>\$</code> (fin línea), <code>gg</code> (inicio archivo), <code>G</code> (fin archivo).
Insertar	<code>i</code> o <code>a</code>	Escritura convencional de texto.
Comando/ Ex	<code>:</code> desde Normal	<code>:w</code> (guardar), <code>:q</code> (salir), <code>:wq</code> (guardar y salir), <code>:q!</code> (forzar salida sin guardar).

Neovim y el concepto de Fork

Neovim es un *fork* de Vim creado en 2014. Un **fork** en software libre ocurre cuando un grupo de desarrolladores toma una copia del código fuente original para continuar su desarrollo de forma independiente, buscando resolver limitaciones de arquitectura, agregar soporte para lenguajes modernos como Lua o ejecutar tareas de forma asíncrona.

6. Permisos del Sistema y Administración (chmod, umask, sudo)

Cada archivo en Linux posee una cadena de 10 caracteres que define su tipo y permisos para tres categorías de usuarios: Propietario (`u`), Grupo (`g`) y Otros (`o`).

```
# Ejemplo de ls -l:
-rwxr-xr-- 1 andres devteam 4096 Aug 7 10:00 script.sh
|         |         |
|         |         | Permisos de OTROS (o): r-- (Solo lectura)
|         |         | Permisos de GRUPO (g): r-x (Lectura y ejecución)
|         |         | Permisos de PROPIETARIO (u): rwx (Lectura, escritura y ejecución)
|         |         | Tipo de archivo (- = regular, d = directorio, l = enlace)
```

Notación Octal de Permisos

- `r` (Lectura) = 4 | `w` (Escritura) = 2 | `x` (Ejecución) = 1
- 755** = `rwxr-xr-x` (Propietario todo; Grupo y Otros lectura/ejecución). Ideal para scripts y directorios.
- 644** = `rw-r--r--` (Propietario lectura/escritura; resto solo lectura). Ideal para archivos planos.

```
chmod +x script.sh      # Añade permiso de ejecución a todos.
chmod 755 script.sh     # Aplica permisos 755 explícitamente.
chmod -R 644 /var/www/html # Aplica recursivamente a toda una carpeta.
```

Máscara de Permisos (`umask`)

La `umask` define qué permisos se **restan** por defecto al crear nuevos archivos o carpetas. Los permisos base de partida son `666` para archivos y `777` para directorios.

- Si `umask` vale `0022`:
 - Nuevo directorio: $777 - 022 = 755$ (`rwxr-xr-x`).
 - Nuevo archivo: $666 - 022 = 644$ (`rw-r--r--`).

7. Procesos, Control de Espacio y Trabajos (Jobs)

Gestión de Espacio en Disco

- `df -h`: ****Disk Free****. Muestra la capacidad total y espacio disponible en todos los puntos de montaje montados en formato legible (GB/MB).
- `du -sh`: ****Disk Usage****. Muestra el tamaño total consumido por una carpeta.

```
du -sh * | sort -hr # Muestra el tamaño de cada elemento ordenado de mayor a menor.
```

Gestión de Procesos y Trabajos

```
ps aux # Lista detallada de todos los procesos del sistema.
top # Monitor interactivo en tiempo real (M = memoria, P = CPU, q = salir).
kill -9 <PID> # Envía la señal SIGKILL para terminar de forma forzada un proceso por su ID.
```

Manejo de Jobs (Segundo y Primer Plano):

- `sleep 100 &`: El símbolo `&` envía el comando a ejecutarse en segundo plano (Background).
- `Ctrl + Z`: Pausa la ejecución del comando actual en consola y lo coloca en estado *Stopped*.
- `jobs`: Lista los trabajos activos asignados a la sesión de terminal actual.
- `bg %1`: Reanuda el trabajo nº 1 en segundo plano.
- `fg %1`: Trae el trabajo nº 1 al primer plano (Foreground).

8. Programación de Scripts en Bash (Shell Scripting)

Un script es un fichero ejecutable que contiene instrucciones de Bash e incorpora estructuras de control lógicas.

Estructura Inicial y Shebang

La primera línea obligatoria debe incluir el **Shebang** (`#!/bin/bash`), que especifica la ruta del intérprete que ejecutará el código.

```
#!/bin/bash
# Script interactivo de ejemplo

read -p "Introduce tu usuario: " nombre_usuario
read -s -p "Introduce clave: " password # -s oculta la escritura
echo ""
echo "Bienvenido, $nombre_usuario"
```

Parámetros Posicionales Especiales

- `$0`: Nombre del script en ejecución.
- `$1, $2, $3...`: Argumentos pasados al script desde la línea de comandos (ej. `./script.sh arg1 arg2`).
- `$#`: Número total de argumentos recibidos.
- `$@`: Lista de todos los argumentos como un conjunto.
- `$?`: Código de estado de retorno de la última orden ejecutada (0 = éxito, >0 = error).

Estructuras Condicionales y Comparadores

Es muy recomendable utilizar la sintaxis moderna de corchetes dobles `[[ ... ]]` por ser más segura y flexible.

Comparador Numérico	Comparador de Texto	Significado
<code>-eq</code>	<code>==</code>	Igual a (Equal)
<code>-ne</code>	<code>!=</code>	No es igual / Distinto (Not Equal)
<code>-gt</code>	N/A	Mayor que (Greater Than)
<code>-ge</code>	N/A	Mayor o igual que (Greater or Equal)
<code>-lt</code>	N/A	Menor que (Less Than)
<code>-le</code>	N/A	Menor o igual que (Less or Equal)

Evaluación de Archivos y Operadores Lógicos:

- `[[ -f "$ruta" ]]`: Verdadero si la ruta existe y es un archivo regular.
- `[[ -d "$ruta" ]]`: Verdadero si existe y es un directorio.
- `[[ -x "$ruta" ]]`: Verdadero si el archivo existe y tiene permiso de ejecución.
- `[[ -z "$texto" ]]`: Verdadero si la cadena está vacía.
- `[[ -n "$texto" ]]`: Verdadero si la cadena NO está vacía.
- Operadores: `&&` (AND), `||` (OR), `!` (NOT).

```
if [[ $edad -ge 18 && $licencia == "si" ]]; then
    echo "Acceso autorizado."
elif [[ ! -f "/etc/config.conf" ]]; then
    echo "Falta archivo de configuración."
else
    echo "Acceso denegado."
fi
```

Bucles (For, While, Until)

```
# Bucle FOR recorriendo un rango
for i in {1..5}; do
    echo "Iteración $i"
done

# Bucle WHILE (mientras se cumpla la condición)
contador=1
while [[ $contador -le 3 ]]; do
    echo "Contador: $contador"
    contador=$((contador + 1))
done
```

Funciones Profesionales en Bash

Aplica siempre la palabra clave `local` para evitar contaminación de ámbito global.

```

calcular_total() {
    local precio=$1
    local impuesto=$2
    local resultado=$((precio + impuesto))

    echo $resultado # Devuelve datos por salida estándar
}

# Captura de la salida de la función:
total_pagar=$(calcular_total 100 21)
echo "Total a pagar: $total_pagar"

```

9. Automatización con Tareas Programadas (Cron Jobs)

El Demonio `crond` ejecuta tareas desatendidas basándose en las especificaciones del archivo `crontab`.

```

crontab -e # Edita las tareas del usuario actual.
crontab -l # Muestra la lista de tareas activas.

```

Sintaxis del Crontab (Las 5 Estrellas)

```

* * * * * /ruta/absoluta/al/script.sh
| | | | |
| | | | | Día de la Semana (0-6, 0=Domingo)
| | | | | Mes (1-12)
| | | | | Día del Mes (1-31)
| | | | | Hora (0-23)
| | | | | Minuto (0-59)

```

⚠ Trampas Clave en Cron Jobs

- 1. PATH Reducido:** Cron no carga tus variables de entorno interactivas. ****Usa siempre rutas absolutas**** (ej. `/usr/bin/python3` en lugar de `python3`).
- 2. Ausencia de Terminal:** Redirige la salida estándar y errores a un archivo de registro para poder auditarlo:

```
0 3 * * * /home/user/backup.sh >> /home/user/cron.log 2>&1
```